

patent_cpc_API — File Map & Descriptions

Structure Map

patent_cpc_API/

main.py	# App entry point - startup, KG init, engine wiring
config/ pipeline_config.yaml domains_schema.json	# All phase thresholds, weights, model names # Domain→CPC mappings, invalid patterns, visual families
core/ engine.py runs phases 1A→5B in sequence state_manager.py shared object across all phases knowledge_graph.py	# Pipeline orchestrator - # PipelineState dataclass - # Singleton wrapper - loads CPCKnowledgeGraph once at start
shared/ base_phase.py cpc_filters.py used in 2B and 3A	# BasePhase ABC - every phase inherits run(state) → dict # is_allocatable(), filter_allocatable() -
api/ models.py router.py	# Pydantic request/response schemas # FastAPI routes: POST /classify, GET /health
phase1a_extraction/ extractor.py prompts/__init__.py	# LLM multi-pass extraction of terms, core function, domain # Re-exports pass A/B/C/D prompts + reconciliation
phase1b_audit/ auditor.py builds anchor_families + kill_log prompts/__init__.py	# Claim-to-domain forensic audit - # Re-exports phase1_2_audit_prompt
phase1c_character/ characterizer.py	# TCR score + role classification (CORE_TECH/SYSTEM/APPL)
phase2a_routing/ router.py	# Family-level routing: embedding + KG + anchor blend → 1
phase2b_expansion/ expander.py	# KG/XML subgroup expansion + non-allocatable pre-filter

phase2c_scoring/ scorer.py	# BM25 + semantic RRF scoring + family-level normalisation
phase2d_filter/ filter.py	# Top-N filter + anchor promotion + preserves all_raw_candidates
phase3a_constraints/ constraints.py	# Decision-tree rule layer: domain boosts, invalid-class
phase3b_validation/ validator.py	# Cross-domain validation: anchor boosts, anti-collapse
phase4a_consolidation/ consolidator.py prompts/__init__.py	# Clusters candidates into hypotheses (family, coherence) # Re-exports phase4_sanity_check_prompt
phase4b_resolution/ resolver.py prompts/__init__.py	# Scores hypotheses on 3 axes → picks winner + Tri-Pillar # Re-exports phase35_tiebreaker_prompt
phase5a_consistency/ consistency.py prompts/__init__.py	# LLM sanity check on Phase 4B primary before final label # Re-exports phase7_consistency_prompt
phase5b_labeling/ labeler.py prompts/__init__.py	# Role labeling (CORE/SUPPORT/CONTEXT) + LLM justification # Re-exports phase8_labeling_prompt

Phase Groups

Group 1 — Signal Extraction

Reads the patent text. No CPC database access yet.

Phase	File	Method	What it does
1A	phase1a_extractor.py	IdMn/extractor.py	Extracts technical terms, core function, technical object, inventive step, domain signals
1B	phase1b_auditor.py	IdMn/auditor.py	Audits which CPC families are actually evidenced in the claims → anchor_families, kill_log

Phase	File	Method	What it does
1C	phase1c_characterize.py	IdTM / characterize Deterministic	Computes TCR (Technology Computation Ratio) and classifies the patent's role

Group 2 — Candidate Search

Queries the CPC database. Produces up to 500 scored subgroup candidates.

Phase	File	Method	What it does
2A	phase2a_routing.py	Eng/Router KG + Anchors	Ranks CPC families (4-char) → family_scores used later for normalisation
2B	phase2b_expansion.py	Decision/Expander	Expands each family into subgroups via KG hierarchy + XML titles
2C	phase2c_scoring.py	BM25/Scorer Semantic + RRF	Scores every subgroup, then normalises per-family so volume doesn't dominate
2D	phase2d_filtering.py	Decision/Filter	Keeps top-N candidates; preserves full raw pool for Phase 4B title lookup

Group 3 — Constraint Validation

Applies deterministic rules. No new database queries.

Phase	File	Method	What it does
3A	phase3a_constraints.py	Decision/Constraint	Decision-tree: domain boosts, invalid-class penalties, functional boosting
3B	phase3b_validation.py	Decision/Validator	Cross-domain validation: anchor confirmed → boost; no context → penalty

Group 4 — Hypothesis

Clusters candidates into competing hypotheses, then picks one winner.

Phase	File	Method	What it does
4A	<code>phase4a_consolidation.py</code>	<code>Deduction/consolidation.py</code>	Groups candidates by family into hypotheses with coherence + support_weight
4B	<code>phase4b_resolution.py</code>	<code>Deduction/resolver.py</code>	Scores hypotheses on phase4 (0.5) + functional_alignment (0.3) + technical_coverage (0.2) → Tri-Pillar

Group 5 — Output

Validates and labels the final recommendation.

Phase	File	Method	What it does
5A	<code>phase5a_consistency.py</code>	<code>Isch/consistency.py</code>	Sanity checks the Phase 4B primary against the full candidate pool
5B	<code>phase5b_labeling.py</code>	<code>Link/labeler.py</code>	Assigns CORE/SUPPORT/CONTEXT roles + writes professional justification text

Core Files

`core/state_manager.py` — `PipelineState`

Single dataclass that flows through every phase. Each phase reads what it needs and writes its output back as a dict (e.g. `state.phase2c = scorer.run(state)`). No global variables — all inter-phase communication goes through this object.

`core/engine.py` — `CPCPipelineEngine`

Instantiates all 13 phase runners at startup. On each request, creates a fresh `PipelineState`, runs the sequence 1A→5B, records errors/warnings, and returns `state.to_dict()`. Phase failures are caught and logged — the pipeline always returns whatever it completed.

`core/knowledge_graph.py` — `KnowledgeGraphSingleton`

Wraps the `CPCKnowledgeGraph` as a singleton. Loads from disk cache on first `get()` call; rebuilds from XML if cache is stale. All phases that need the KG

receive the same object instance.

shared/cpc_filters.py — Allocatable guards

`is_allocatable(symbol)` returns `True` only when the symbol contains / AND does not match the cross-reference pattern `^[A-Z]\d{2}[A-Z]\d{4}/`. Applied in Phase 2B (after expansion) and Phase 3A (before constraint rules).

Config Files

config/pipeline_config.yaml

Every numeric threshold in the pipeline is defined here. Phase code reads its own section (e.g. `phase2c_scoring.rrf_k`) via the `cfg` dict injected at startup. Changing a threshold never requires touching phase code.

config/domains_schema.json

- `domain_to_cpc` — maps domain keywords to CPC family lists
- `invalid_patterns` — per-domain CPC prefixes that should be penalised
- `visual_families` — families excluded from Pillar 3 when primary is G10L
- `cross_ref_pattern` — regex to detect non-allocatable cross-reference codes